

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4

ШАБЛОНИ «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY»

Виконав
студент групи ІА–22:
Артамонов Дмитро
Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості 3
Реалізувати не менше 3-х класів відповідно до обраної теми 5
Реалізувати один з розглянутих шаблонів за обраною темою. 6
Висновок 8

Тема: шаблони «singleton», «iterator», «proxy», «state», «strategy».

Мета: ознайомитися з шаблонами проектування «Singleton», «Iterator», «Proxy», та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

..15 E-mail клієнт (singleton, builder, decorator, template method, interpreter, SOA)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти

---

pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Короткі теоретичні відомості

Шаблони проєктування представляють собою стандартизовані підходи до вирішення типових задач, які часто зустрічаються при створенні інформаційних систем. Вони включають в себе опис проблеми, ефективне рішення та рекомендації щодо застосування в різних контекстах. Завдяки загальновизнаним назвам, шаблони легко розпізнаються та багаторазово використовуються.

Ключовою особливістю роботи з шаблонами є ретельне моделювання предметної області, що дозволяє точно сформулювати проблему і підібрати найкращий шаблон. Використання таких рішень дає розробнику ряд переваг: вони допомагають зробити систему більш організованою, зрозумілою для вивчення та легкою у розширенні. Крім того, це підвищує гнучкість системи до змін і сприяє її простішій інтеграції з іншими рішеннями.

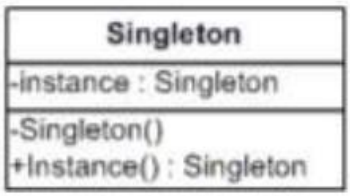
Таким чином, шаблони проєктування є перевіреними часом інструментами, які широко застосовуються в різних організаціях і проєктах для побудови ефективної архітектури в заданих умовах.

Навіщо використовувати шаблони ?

Шаблони проєктування сприяють зменшенню часу і зусиль на створення архітектури, надаючи системі важливі властивості, як-от гнучкість і адаптованість, а також спрощують її підтримку. Їх застосування полегшує комунікацію в команді, оскільки спільне розуміння шаблонів робить архітектуру зрозумілою для всіх учасників проєкту. Шаблони дозволяють уникнути "винаходу велосипеда", використовуючи перевірені практики, визнані спільнотою розробників, що економічно вигідно для бізнесу. Водночас їх використання повинно бути обґрунтованим, адже вони не є універсальним рішенням для всіх ситуацій.

Шаблон “Singleton”

Структура:



Призначення патерну Singleton

Singleton — це клас, що гарантує створення лише одного екземпляра, який централізовано керує ресурсами чи операціями. Його застосування доцільне, коли потрібно уникнути надлишкових об'єктів і забезпечити контроль над доступом до спільних даних або процесів.

Проблема

Singleton потрібен, якщо:

- Необхідно обмежити кількість об'єктів класу до N екземплярів.
- Потрібно контролювати всі операції, що проходять через один об'єкт.

Рішення

Singleton використовується для зберігання унікального файлу конфігурацій, що виключає створення зайвих екземплярів. Також він підходить для управління єдиним сеансом зв'язку між системами, реалізуючи контроль над запитами й відповідями.

Реалізувати не менше 3-х класів відповідно до обраної теми

Структура проекту з реалізованими класами зображена на рисунку 1.

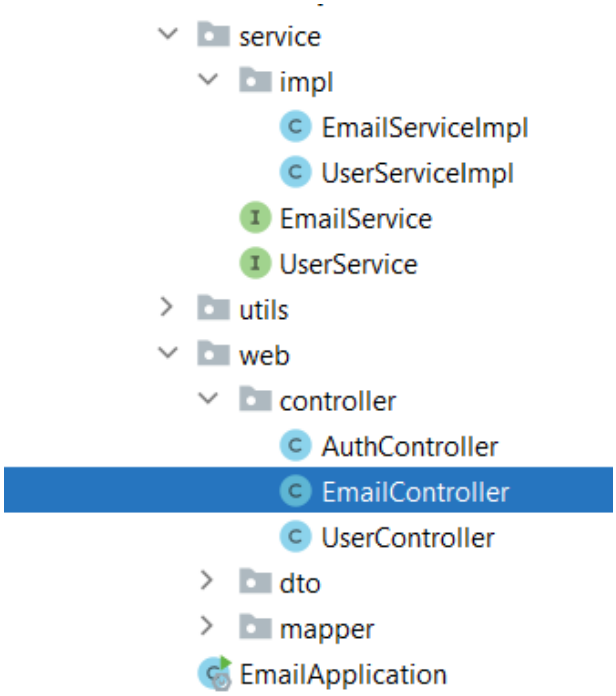


Рисунок 1 – структура класів

Опис класів

В ході виконання лабораторної роботи були реалізовані наступні класи:

1. **AuthController –** це клас, який обробляє HTTP-запити, пов'язані зі створенням нових користувачів, і призначений для взаємодії з клієнтами, які ще не зареєстровані.
2. **EmailController -** це клас, який обробляє HTTP-запити, пов'язані з відправленням електронних листів. Його призначення полягає у забезпеченні функціоналу для клієнтів, які хочуть скористатися сервісом відправки email-повідомлень.
3. **UserController -** це клас, який обробляє HTTP-запити, пов'язані з управлінням обліковими записами користувачів. Його основне завдання

— забезпечити функціонал додавання нових поштових скриньок до облікового запису користувача.

4. **UserServiceImpl -** це сервіс, який керує створенням користувачів, активацією облікових записів та додаванням поштових скриньок до профілів. Він забезпечує перевірку даних, кодування паролів, а також інтеграцію з відповідними репозиторіями для збереження змін.
5. **EmailServiceImpl -** це сервіс, який відповідає за відправлення електронних листів, використовуючи облікові записи користувачів та їх конфігурації. Він виконує налаштування параметрів SMTP-з'єднання відповідно до конфігурації поштової скриньки та забезпечує автентифікацію за допомогою токенів.

Реалізувати один з розглянутих шаблонів за обраною темою.
Для виконання даного пункту лабораторної роботи було реалізовано шаблон «Одинак» для класу EmailServiceImpl, який забезпечує в системі лише один об’єкт цього класу.

```
@Service
public class EmailServiceImpl implements EmailService {
    2 usages
    private final MailBoxRepository mailBoxRepository;
    3 usages
    private EmailServiceImpl emailService;
    1 usage  new *
    private EmailServiceImpl(MailBoxRepository mailBoxRepository) {
        this.mailBoxRepository = mailBoxRepository;
    }
    no usages  new *
    @Autowired
    public EmailServiceImpl getInstance(MailBoxRepository mailBoxRepository) {
        if (emailService == null) {
            emailService = new EmailServiceImpl(mailBoxRepository);
        }
        return emailService;
    }
```

Рисунок 2 – реалізований шаблон «Одинак»

У класі **EmailServiceImpl** конструктор є приватним, що забороняє його виклик поза межами самого класу. Створення екземпляра відбувається

виключно через метод getInstance. Цей метод забезпечує, що об’єкт створюється лише один раз: під час першого виклику ініціалізується новий екземпляр класу, а всі наступні виклики повертають вже створений екземпляр. Такий підхід гарантує дотримання принципу Singleton і унеможливлює дублювання екземплярів класу.

Проблема, яку допоміг вирішити шаблон «Одинак»

Застосування патерну **Singleton** вирішує проблему створення декількох екземплярів сервісу, які могли б конфліктувати між собою при доступі до поштових скриньок або їх конфігурацій. Наприклад, якщо кожен запит ініціює новий екземпляр класу, це може призвести до зайвого споживання ресурсів і неконсистентної роботи з даними.

Переваги застосування патерну «Одинак»

1. **Єдиний доступ до логіки відправки листів**:
   Гарантовано, що всі запити використовують один і той самий екземпляр класу, що забезпечує централізоване управління операціями відправки електронних листів.
2. **Зменшення споживання ресурсів**:
   Оскільки створюється лише один екземпляр EmailServiceImpl, зменшується навантаження на пам'ять та інші ресурси, пов'язані з ініціалізацією.
3. **Безпека даних і стану**:
   Забезпечується консистентний стан об'єктів, наприклад, конфігурацій поштових скриньок. Це виключає можливість одночасного доступу до некоректно налаштованих екземплярів.

Діаграма класів для паттерну «Одинак»

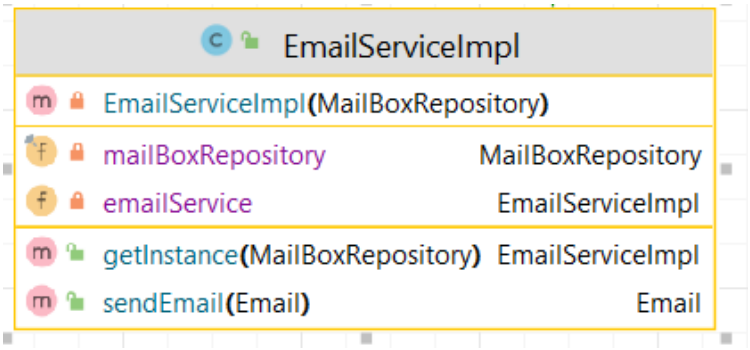


Рисунок 3 – діаграма класів

Посилання на репозиторій з кодом: https://github.com/qjsoq/Email-API

Висновок

У ході виконання лабораторної роботи було реалізовано патерн проєктування Singleton для сервісу відправки електронних листів. Застосування цього патерну дозволило забезпечити створення лише одного екземпляра класу EmailServiceImpl, що гарантує централізоване управління логікою роботи з поштовими скриньками. Завдяки Singleton вдалося уникнути дублювання об'єктів, що зменшило споживання ресурсів та підвищило консистентність системи. Реалізація цього патерну зробила сервіс більш структурованим і легким у підтримці, спростивши інтеграцію з іншими компонентами.